

G03 / PROTECTED ANSWERS

G03 REVIEW A

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 3

G03 Review A: worked implementation and reasoning 3

Make the duplicate policy explicit 3

Expected fixture result 3

Read the algorithm before the code 3

Complete reference 3

Bash wrapper 5

Check the attempt 5

A worked Git and clean-clone route 5

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

G03 Review A: worked implementation and reasoning

Open only after saving Gate A. This is one defensible reference; a different implementation can pass the same stated contract.

Make the duplicate policy explicit

The first source occurrence reserves an ID, even when that row is later rejected. Every later occurrence is rejected. Amounts/costs must be finite and nonnegative; cartons must be positive whole numbers. These are the reference's declared rules. The assessment accepts another explicitly justified duplicate policy only if its controls are recomputed and reconciled consistently.

Expected fixture result

A: raw=8, accepted=4, rejected=4. Accepted Completed rows GA01 and GA03 give East=1200.00 and North=900.00; total=2100.00. GA02 Pending and GA08 Cancelled are accepted but excluded from Completed revenue. Missing ID, Unknown status, second GA02 and bad amount are rejected.

Read the algorithm before the code

Track seen keys separately from accepted rows. Each source row goes to exactly one output: accepted or rejected with its original text and reason. Decimal avoids binary floating-point drift for these money calculations. Group totals include only the requested status. Sorted JSON keys and totals make identical reruns comparable. The input file is only read.

Complete reference

Copy `validate_a.py` to `validate.py` only in your review-practice repository. It uses the Python standard library. Create the `.venv` in that same repository as the wrapper expects. The final two positional arguments are the input CSV and output JSON; the gate variant is fixed by this file.

```
"""Worked G03 A/B reference. Open only after the matching attempt.
```

```
Source order is authoritative for duplicates: first ID reserves the key,
including an invalid first occurrence. A later occurrence is rejected.
This is a stated assessment policy, not a universal business deduplication rule.
"""
```

```
import argparse
import csv
import json
from collections import Counter, defaultdict
from decimal import Decimal, InvalidOperation
from pathlib import Path

def number(text, positive=False, integer=False):
    try:
        value = Decimal(text)
    except (InvalidOperation, ValueError, TypeError):
```

```

        raise ValueError("invalid_number") from None
    if not value.is_finite() or value < 0 or (positive and value == 0):
        raise ValueError("invalid_number")
    if integer and value != value.to_integral_value():
        raise ValueError("not_whole_cartons")
    return value

def validate(rows, variant="A"):
    key, area = ("order_id", "region")
    allowed = {"Completed", "Pending", "Cancelled"}
    include = "Completed"
    seen, accepted, rejected = set(), [], []
    totals, statuses = defaultdict(Decimal), Counter()
    for line, row in enumerate(rows, 2):
        identifier = (row.get(key) or "").strip()
        reason = None
        if not identifier:
            reason = "missing_id"
        elif identifier in seen:
            reason = "duplicate_id"
        else:
            seen.add(identifier)
        if reason is None and row.get("status") not in allowed:
            reason = "invalid_status"
        value = None
        if reason is None:
            try:
                value = number(row["amount"])
            except (ValueError, KeyError) as exc:
                reason = str(exc)
        if reason:
            rejected.append({"source_line": line, "row": row, "reason": reason})
            continue
        accepted.append(**row, key: identifier, "validated_value": str(value))
        statuses[row["status"]] += 1
        if row["status"] == include:
            totals[row[area]] += value
    assert len(rows) == len(accepted) + len(rejected)
    assert len({r[key] for r in accepted}) == len(accepted)
    return {
        "variant": variant,
        "raw_rows": len(rows),
        "accepted_rows": len(accepted),
        "rejected_rows": len(rejected),
        "accepted_status_counts": dict(sorted(statuses.items())),
        "totals": {k: format(v, ".2f") for k, v in sorted(totals.items())},
        "accepted": accepted,
        "rejected": rejected,
    }

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("source", type=Path)
    parser.add_argument("output", type=Path)
    args = parser.parse_args()
    with args.source.open(newline="", encoding="utf-8-sig") as f:
        result = validate(list(csv.DictReader(f)), "A")
    args.output.parent.mkdir(parents=True, exist_ok=True)
    # A disposable report can be replaced. The input file is only read.
    if args.output.resolve() == args.source.resolve():
        raise ValueError("output must not overwrite the source")
    args.output.write_text(json.dumps(result, indent=2, sort_keys=True) + "\n")

```

Bash wrapper

Save this as `run_a.sh` next to `validate.py`. Run it with `bash`; executable-bit setup is not required. Relative input/output paths are relative to the terminal's current folder. The wrapper itself locates its Python interpreter and script relative to its own location. A missing file returns exit code 2; a Python failure propagates because `set -e` is active.

```
#!/usr/bin/env bash
set -euo pipefail
here="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd)"
input="${1:?Usage: bash run_a.sh INPUT_CSV [OUTPUT_JSON]}"
output="${2:-output/report.json}"
[[ -f "$input" ]] || { echo "Missing input: $input" >&2; exit 2; }
"$here/.venv/bin/python" "$here/validate.py" "$input" "$output"
```

Check the attempt

Run twice to two output files and compare them. Rename the input temporarily in a disposable copy and prove the wrapper fails without publishing a new report. Introduce a repeated key in a copied fixture and explain its rejected row. Keep the unchanged original.

A worked Git and clean-clone route

The following commands operate only inside a new disposable gate-practice repository. They explain the sequence; they are not evidence that you personally ran it. Use your actual configured Git identity. Never paste a real secret into a demonstration.

First create `.gitignore` containing `.venv/`, `.env` and `output/`. Create `.env.example` with `INPUT_FILE=` and no credential. Put `validate.py`, the Bash wrapper, `README.md` and the harmless fixture under source control. The standard-library validator needs no third-party package. The `README` must name Python, Bash, the working folder and the exact wrapper command.

```
git init -b main
python3 -m venv .venv
git add .
git commit -m "Add validator, fixture and run instructions"
git switch -c feature/policy
```

For a controlled conflict, commit one validation-policy line in `policy.txt` on `main` before branching. On `feature/policy` replace that same line with one alternative and commit. Switch to `main`, replace the original line with a different alternative and commit. Now merge `feature/policy`. Git reports a conflict because both branches changed the same part since their common ancestor. Open `policy.txt`, remove the marker lines and write the intended final policy. A conflict marker is not valid policy text.

```
git status
git add policy.txt
git commit -m "Resolve validation policy by intended rule"
```

For Gate A put the deliberate conflict in the `README` run instruction instead; resolve the command to the wrapper interface actually used in A. For Gate B use branch `feature/hub-controls` and the validation-policy line required by B. Do not merely accept an entire side without reading its meaning.

An uncommitted mistake and a committed mistake have different recovery evidence. Make the deliberate uncommitted change only in a disposable practice file. Inspect `git diff` first; then `git`

restore --worktree -- policy.txt discards that uncommitted change. To reverse a deliberate committed error, first ensure the working tree is clean; git revert HEAD creates a new inverse commit. Keep git log --oneline and the before/after file to explain both operations. Do not use reset --hard as the shortcut.

For an honest offline remote, run the next commands from the parent of your newly created gate repository; replace g03-order-quality with your actual repository folder for B. This is a local bare remote, not GitHub.

```
git init --bare gate-remote.git
git -C g03-order-quality remote add origin ../gate-remote.git
git -C g03-order-quality push -u origin main
git clone -b main gate-remote.git clean-clone
cd clean-clone
python3 -m venv .venv
```

Run the wrapper using only the cloned README. Compare the resulting JSON with the first run, excluding nothing silently. Identical input and policy must produce identical sorted JSON. The raw input hash must still match. Record OFFLINE LOCAL REMOTE if this was your route; record GitHub push/fetch only if you actually performed it.

To demonstrate fetch, make and push a harmless README commit in clean-clone. Return to the original repository; save git rev-parse HEAD and its README before git fetch origin. Fetch updates origin/main, but leaves HEAD and the working README unchanged. Compare git rev-parse HEAD with git rev-parse origin/main, then inspect git log --oneline --all. Integrating the commit is a separate merge/rebase decision.

.gitignore affects future untracked files; it neither erases an already tracked secret nor revokes a copied credential. If a real credential was exposed, revoke/rotate it at the provider first, then remove it from tracked content and follow the host's history-cleanup procedure. The course example must use dummy text only.